**Zapret действительно распознаёт MTProto**:

```
static const char *l7proto_name[] =
{"all","unknown","known","http","tls","quic","wireguard","dht","discord","stun","xmpp","
dns","mtproto"};
```

```
 "mtproto_initial"};
```

## Как работает определение MTProto

```
bool IsMTProto(const uint8_t *data, size_t len)
{
    if (len>=64)
    {
        // this way requires only one AES instead of 4
        uint8_t decrypt[16], iv[16];
        aes_context ctx;

        memcpy(iv, data+40, 16);
        ctr_add(iv,3);
        if (!aes_setkey(&ctx, AES_ENCRYPT, data+8, 32) && !aes_cipher(&ctx, iv,
decrypt))
        {
            *((uint32_t*)(decrypt+8)) ^= *((uint32_t*)(data+56));
            return !memcmp(decrypt+8,"\xEF\xEF\xEF\xEF",4);  // сигнатура MTProto2
        }
    }
    return false;
}
```

Определяется через **AES дешифровку** — проверяется магическая сигнатура `0xEFEFEFEF`.

## Ограничения

```
        else if (ctrack && (ctrack->pos.seq_last - ctrack->pos.seq0)==1 &&
IsMTProto(dis->data_payload, dis->len_payload))
        {
            DLOG("packet contains telegram mtproto2 initial\n");
            // mtproto detection requires aes. react only on the first tcp data packet.
do not detect if ctrack unavailable.
            l7payload = L7P_MTPROTO_INITIAL;
```

1. Определяется **только на первом TCP пакете с данными**
2. Требуется **conntrack** (ctrack)
3. Последующие пакеты будут **unknown**

## Важно для фильтрации!

**MTProto НЕ содержит hostname!** Это значит:

| Фильтрация | Работает? |
|---|---|
| `--filter-l7=mtproto` | ✅ Да |
| `--payload=mtproto_initial` | ✅ Да |
| `--hostlist=...` | ❌ **Нет** (нет hostname) |
| `--ipset=telegram_ips.txt` | ✅ Да |

## Пример использования

```
nfqws2 --lua-init=@zapret-lib.lua --lua-init=@zapret-antidpi.lua \
  --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=fake:blob=0x00000000:repeats=3
```

Или с ipset для большей точности:

```
nfqws2 \
  --ipset=telegram_ips.txt \
  --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=multisplit:pos=32
```

hostlist бесполезен — только ipset или фильтрация по l7 proto.

Ключевое условие в коде:

```
        else if (ctrack && (ctrack->pos.seq_last - ctrack->pos.seq0)==1 &&
IsMTProto(dis->data_payload, dis->len_payload))
        {
            DLOG("packet contains telegram mtproto2 initial\n");
            // mtproto detection requires aes. react only on the first tcp data packet.
do not detect if ctrack unavailable.
            l7payload = L7P_MTPROTO_INITIAL;
            if (l7proto == L7_UNKNOWN)
            {
                l7proto = L7_MTPROTO;
                if (ctrack->l7proto == L7_UNKNOWN) ctrack->l7proto = l7proto;
            }
        }
```

## Когда MTProto → unknown

**Условие детекции:**

```
(ctrack->pos.seq_last - ctrack->pos.seq0) == 1
```

Это значит: **только первый TCP пакет с данными**.

```
TCP соединение MTProto:
──────────────────────────────────────────────────────────────────────────►

[SYN]     [SYN+ACK]     [ACK]     [DATA #1]          [DATA #2]     [DATA #3] ...
  │           │           │           │                  │             │
  │           │           │           │                  │             │
  │           │           │           └── seq_last-seq0=1  └─────────────┴── seq_last-
seq0 > 1
  │           │           │               payload=mtproto_initial
payload=unknown
  │           │           │               proto=mtproto                  proto=mtproto
(сохранён)
  │           │           │
  └───────────┴───────────┴── payload=empty
```

## Разница l7proto vs payload

| Пакет | `--filter-l7` (proto) | `--payload` |
|---|---|---|
| SYN, SYN+ACK, ACK | unknown | empty |
| **1-й пакет с данными** | **mtproto** | **mtproto_initial** |
| 2-й пакет с данными | mtproto | **unknown** |
| 3-й пакет с данными | mtproto | **unknown** |
| ... | mtproto | **unknown** |

**l7proto** (протокол соединения) сохраняется в conntrack и остаётся `mtproto` для всего соединения.

**payload** (тип содержимого пакета) — `mtproto_initial` только для первого пакета, потом `unknown`.

## Практический вывод

```
# Сработает только на первом пакете с данными
--filter-l7=mtproto --payload=mtproto_initial --lua-desync=fake

# Сработает на всех пакетах MTProto соединения (включая unknown)
--filter-l7=mtproto --payload=all --lua-desync=multisplit

# Или без фильтра payload (по умолчанию known, mtproto_initial входит)
--filter-l7=mtproto --lua-desync=fake
```

**Да, можно!** Множественные payload поддерживаются через запятую:

```
static bool parse_l7p_list(char *opt, uint64_t *l7p)
{
    char *e, *p, c;
    t_l7payload payload;

    for (p = opt, *l7p = 0; p; )
    {
        if ((e = strchr(p, ',')))  // парсинг через запятую
        {
            c = *e;
            *e = 0;
```

## Правильный синтаксис

```
--payload=mtproto_initial,unknown
```

Это сработает на:

- Первом пакете MTProto ( `mtproto_initial` )
- Всех последующих пакетах ( `unknown` )

## Пример для MTProto

```
nfqws2 \
  --filter-l7=mtproto \
  --payload=mtproto_initial,unknown \
  --lua-desync=fake:blob=0x00000000:repeats=2
```

Или если нужны разные стратегии для разных payload:

```
nfqws2 \
  --filter-l7=mtproto \
  --payload=mtproto_initial \
    --lua-desync=fake:blob=0x00000000:repeats=3 \
  --payload=unknown \
    --lua-desync=multisplit:pos=16
```

## Доступные payload типы

Из кода `protocol.c` :

```
empty, http_req, http_resp, tls_client_hello, tls_server_hello,
tls_app_data, quic_initial, quic_0rtt, wireguard_initiation,
wireguard_response, wireguard_cookie, dht, discord_ip_discovery,
stun, xmpp_stream, xmpp_starttls, dns_req, dns_resp, mtproto_initial,
unknown, known, all
```

## Лучший подход для MTProto для обхода блокировок

**Только первый пакет имеет значение!** Вот почему:

### Как DPI детектит MTProto

1. **По первому пакету** — есть характерная структура (64+ байт, AES сигнатура)
2. **По IP серверов** — известные диапазоны Telegram
3. После установления соединения — всё зашифровано, DPI не видит ничего

## Рекомендация

```
nfqws2 \
  --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=fake:blob=0x00000000000000000000000000000000:repeats=3
```

**Почему так:**

| Вариант | Плюсы | Минусы |
|---|---|---|
| `payload=mtproto_initial` | Минимум нагрузки, точно по цели | — |
| `payload=mtproto_initial,unknown` | Охватывает всё | Лишняя обработка зашифрованных пакетов |
| `payload=all` | Проще написать | Обрабатывает даже пустые ACK |

## Почему unknown не нужен

После первого пакета MTProto2:

- Трафик **полностью зашифрован**
- Выглядит как **случайные данные**
- DPI **не может** его отличить от обычного шума
- Обработка = **пустая трата CPU**

## Если блокируют жёстко

Если DPI анализирует паттерны (размеры, тайминги), можно добавить:

```
nfqws2 \
  --filter-l7=mtproto \
  --payload=mtproto_initial \
  --lua-desync=fake:blob=0x00000000000000000000000000000000:repeats=3 \
  --lua-desync=multisplit:pos=32
```

## Если блокируют по IP

Тут только VPN/proxy поможет, или использовать MTProxy с obfuscation.

---

**TL;DR:** Используй `--payload=mtproto_initial` — этого достаточно в 99% случаев.